



Anchor V1.1

Security Assessment

June 9th, 2026 — Prepared by OtterSec

Jamie Hill - Daniel

jamie@osec.io

Akash Thota

0x4ka5h@osec.io

Xiang Yin

soreatu@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	3
Findings	4
Vulnerabilities	5
OS-ANC-ADV-00 Possibility of Overwriting CPI Return Data	6
OS-ANC-ADV-01 IDL Ignores Borsh Layout Attributes	7
General Findings	8
OS-ANC-SUG-00 Silent Migration Success Without Persistence	9
OS-ANC-SUG-01 <code>max_len</code> Accepts Non-Integer Literals	10
OS-ANC-SUG-02 Presence of Feature-Gated Panic Paths	11
Appendices	
Vulnerability Rating Scale	12
Procedure	13

01 — Executive Summary

Overview

OtterSec conducted an assessment of the **anchor-V1.1** framework. This assessment was conducted between May 25th and June 3rd, 2026.

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified a vulnerability in PR#4411, which reads live global return data, so a later CPI to the same program may overwrite the original return value while still passing the program-id check ([OS-ANC-ADV-00](#)). Additionally, the IDL generation path in PR#4380 ignores Borsh layout attributes, resulting in Rust serialization and generated TS/client coders disagreeing on the actual byte layout ([OS-ANC-ADV-01](#)).

We also suggested returning an explicit error when the destination owner is not the executing program during migration ([OS-ANC-SUG-00](#)), and recommended restricting literal handling to integer literals when parsing the length attribute ([OS-ANC-SUG-01](#)). Lastly, we advised removing all remaining clippy-denied fallible paths in anchor-syn ([OS-ANC-SUG-02](#)).

02 — Scope

The source code was delivered to us in a git repository at <https://github.com/otter-sec/anchor>.

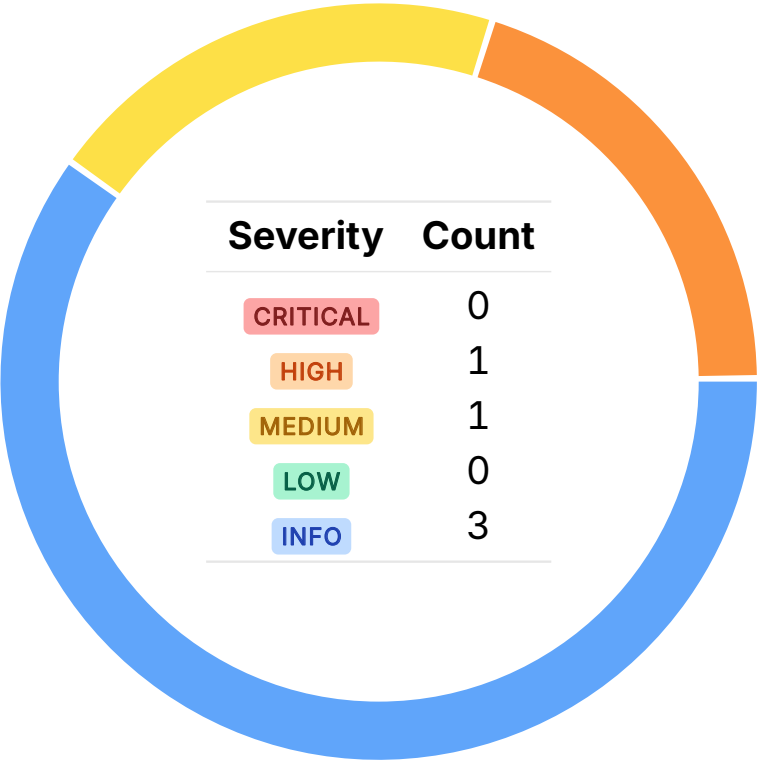
A brief description of the program is as follows:

Name	Description
anchor-V1.1	Review of the Anchor-V1.1 framework.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-ANC-ADV-00	HIGH	RESOLVED ✓	In PR#4411, <code>Return::get()</code> reads live global return data, so a later CPI to the same program may overwrite the original return value while still passing the program-id check.
OS-ANC-ADV-01	MEDIUM	RESOLVED ✓	The IDL generation path in PR#4380 ignores Borsh layout attributes, resulting in Rust serialization and generated TS/client coders to disagree on the actual byte layout.

Possibility of Overwriting CPI Return Data HIGH

OS-ANC-ADV-00

Description

Currently in [PR#4411](#), `Return<T>` stores only the expected `program_id` and calls `get_return_data()` later, so it reads the current global Solana return-data slot rather than the bytes produced at the time of the CPI. If another CPI to the same program executes before `Return::get()` is called, it may overwrite return data while still passing the program-id check, resulting in `get()` deserialising the later CPI's return value as if it came from the original call.

Remediation

Ensure the CPI helper snapshots `(key, data)` immediately after `invoke_signed` succeeds and store the returned bytes inside `Return<T>`, and update `get()` such that it deserializes the stored bytes instead of calling `get_return_data()`.

Patch

Resolved in [PR#4624](#).

IDL Ignores Borsh Layout Attributes MEDIUM

OS-ANC-ADV-01

Description

The updated implementation in [PR#4380](#) results in the serialization derive now forwarding item-level `#[borsh]` attributes to the generated Borsh implementation, but the IDL generation path must apply the same semantic adjustments. If `#[borsh(skip)]` fields are still emitted into the IDL, clients will expect bytes that Borsh does not serialize, resulting in Rust account/instruction layouts to diverge from TS coders. Similarly, when `#[borsh(use_discriminant = true)]` is used, explicit `enum` discriminant values must be preserved in the IDL so non-Rust clients encode/decode the same tag bytes as Borsh. Otherwise, the Rust program and generated IDL clients may disagree on both field presence and `enum` variant tags despite compiling successfully.

Remediation

Ensure the IDL generation path accounts for `#[borsh]` attributes.

Patch

Resolved in [PR#4623](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ANC-SUG-00	<code>Migration::exit</code> in PR#4060 should not return <code>Ok(())</code> when <code>To::owner() != program_id</code> , because the migrated <code>To</code> value cannot be serialized and is silently discarded. This falsely reports migration success while leaving the on-chain account in the old <code>From</code> format.
OS-ANC-SUG-01	PR#3879 broadens <code>#[max_len]</code> literal parsing from integer literals to any literal, allowing invalid values such as floats or strings to reach generated code.
OS-ANC-SUG-02	PR#4291 does not fully enforce its proc-macro panic-path policy because <code>anchor-syn</code> still has feature-gated unguarded <code>unwrap</code> , <code>panic</code> , and unimplemented calls under <code>idl-build</code> and <code>event-cpi</code> .

Silent Migration Success Without Persistence

OS-ANC-SUG-00

Description

In the `Migration::exit` implementation in [PR#4060](#), reaching `MigrationInner::To(to)` implies the account has been migrated in memory and must now be serialized to storage for the migration to take effect. If `To::owner() != program_id`, the current program will skip persistence, leaving the on-chain account in its previous serialised state while returning `Ok()`, thereby falsely implying a successful migration. This should return an error instead, as a migrated `To` value that cannot be written back represents a failed migration.

```
>_ anchor/lang/src/accounts/migration.rs
```

RUST

```
fn exit(&self, program_id: &Pubkey) -> Result<()> {  
    // Check if account is closed  
    if crate::common::is_closed(self.info) {  
        return Ok(());  
    }  
    // Check that the account has been migrated and serialize  
    match &self.inner {  
        MigrationInner::From(_) => {  
            // Account was not migrated - this is an error  
            return Err(ErrorCode::AccountNotMigrated.into());  
        }  
        MigrationInner::To(to) => {  
            // Only persist if the owner is the current program  
            let expected_owner = To::owner();  
            if &expected_owner != program_id {  
                return Ok(());  
            }  
            // Serialize the migrated data  
            let mut data = self.info.try_borrow_mut_data()?;  
            let dst: &mut [u8] = &mut data;  
            let mut writer = BpfWriter::new(dst);  
            to.try_serialize(&mut writer)?;  
        }  
    }  
    Ok()  
}
```

Remediation

Return an explicit error when `To::owner()` is not the executing program.

Patch

Resolved in [PR#4622](#).

max_len Accepts Non-Integer Literals

OS-ANC-SUG-01

Description

The update in [PR#3879](#) modifies `parse_len_arg` to accept any `Expr::Lit`, instead of just integer literals, which broadens `#[max_len]` from integer-only bounds to arbitrary literals such as floats, strings, bytes, or booleans. Those values are then emitted as `(#expr as usize)`, resulting in either poor downstream compiler diagnostics or unintended numeric casts for literals that should never represent byte or element lengths. This weakens the macro's input validation compared to the prior `LitInt` parsing, where only integer literals were accepted as concrete length values.

```
>_ anchor/lang/derive/space/src/lib.rs
```

RUST

```
fn parse_len_arg(item: ParseStream) -> Result<VecDeque<TokenStream2>, syn::Error> {  
    // Parse comma-separated expressions  
    let exprs = item.parse_terminated::<Expr, syn::token::Comma>(Expr::parse)?;  
    let mut result = VecDeque::new();  
  
    // Push them in reverse because get_next_arg() pops from the back  
    for expr in exprs.into_iter().rev() {  
        if !matches!(expr, Expr::Path(_) | Expr::Lit(_)) {  
            return Err(syn::Error::new(  
                expr.span(),  
                "max_len only accepts identifiers, literals, or paths",  
            ));  
        }  
        result.push_back(quote!((#expr as usize)));  
    }  
  
    Ok(result)  
}
```

Remediation

Restrict literal handling to `syn::Lit::Int` to ensure that `max_len` values are compile-time integer lengths, while still allowing identifiers and paths for constants.

Patch

Resolved in [PR#4622](#).

Presence of Feature - Gated Panic Paths

OS-ANC-SUG-02

Description

[PR#4291](#)'s lint hardening is incomplete because `anchor-syn` still has feature-gated panic paths that are only compiled under `idl-build` and `event-cpi`. As a result, `cargo clippy --all-features` continues to fail on unguarded `unwrap`, `panic`, or unimplemented calls in the listed `lang/syn` modules, despite the PR claiming proc-macro panic paths are denied or justified:

1. `lang/syn/src/parser/accounts/event_cpi.rs`
2. `lang/syn/src/idl/accounts.rs`
3. `lang/syn/src/idl/defined.rs`
4. `lang/syn/src/idl/external.rs`
5. `lang/syn/src/idl/program.rs`

Remediation

Remove all remaining clippy-denied fallible paths in `anchor-syn`.

Patch

Resolved in [PR#4631](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.